

Network Anomaly Detection System

Project Progress Report — P1 (Network Engineer + Team Lead)

Field	Details
Student / Role	Elina (P1) — Network Engineer + Team Lead
Project	Network Anomaly Detection System
GitHub Repo	github.com/elinayasmin/network-anomaly-detection
Report Date	May 2026
Weeks Covered	W1 (Planning & Setup) + W2 (API & Topology)
Tasks Completed	3 of 8 fully done Task 4 in progress

Report Contents

#	Section	Status
1	Project Overview & Architecture	✓
2	Team Structure & Dependencies	✓
3	Task 1 — Project Planning (W1)	✓ Done
4	Task 2 — Repository Setup (W1)	✓ Done
5	Task 3 — Flask API + SQLite (W2)	✓ Done
6	Task 4 — Packet Tracer Topology (W2)	■ In Progress
7	Tool Decisions & Rationale	✓
8	Failures & Fixes	✓
9	Interview Answer Framework	✓
10	Remaining Tasks	■

1. Project Overview & Architecture

The Network Anomaly Detection System is a real-time pipeline that captures network traffic, passes it through a machine learning model, and displays alerts on a web dashboard. It functions as a mini Intrusion Detection System (IDS) built entirely from free and open-source tools.

System Pipeline:

Stage	Component	Owner	Tool
1 — Capture	Network topology + traffic data	P1	Cisco Packet Tracer + CICIDS
2 — Data	Label + clean traffic dataset	P2	CICIDS dataset + pandas
3 — Detection	ML anomaly model	P3	scikit-learn
4 — API	REST API + storage	P1	Flask + SQLite
5 — Dashboard	Live alert visualization	P4	Frontend framework

Key Design Decisions:

- **GNS3 → Cisco Packet Tracer:** Switched to master Cisco-specific tools for Cisco interview preparation
- **Scapy → CICIDS Dataset:** Real labeled attack traffic gives ML model stronger training than synthetic data
- **Django → Flask:** Lightweight API — no overhead needed for internal microservice
- **PostgreSQL → SQLite:** Zero server setup — sufficient for academic project scale

2. Team Structure & Dependencies

Person	Role	Key Responsibilities
P1 (Elina)	Network Engineer + Team Lead	GNS3/PT topology, pyshark pipeline, Flask API, GitHub owner
P2	Security / Data Engineer	CICIDS dataset, attack detection rules, security metrics
P3	ML Engineer	Model training, predict() function, real-time inference
P4	Frontend Engineer	Dashboard UI, live anomaly visualization

Critical Dependencies:

Dependency	From → To	Risk if Late
End of W2	P2 → P3: labeled CSV handed off	P3 blocked — cannot train ML model
End of W3	P1 + P3 → P4: pipeline + predict() ready	P4 blocked — dashboard shows no live data

Task Overview (P1):

Task Name	Week	Status	Git Branch	Issues
P1 — Project planning, role allocation, timeline (W1)	W1	Not Started	chore/w1-planning	
P1 — Set up repo, branching strategy, issue labels (W1)	W1	Not Started	chore/w1-repo-setup	
P1 — Integrate backend components + define API contracts (W2)	W2	Not Started	backend/w2-api-contracts	
P1 — Network data capture pipeline (pcap/NetFlow) design (W2)	W2	Not Started	backend/w2-capture-design	
P1 — Backend integration: connect detection outputs to API + storage (W3)	W3	Not Started	backend/w3-integration	
P1 — Finalize system architecture diagram + component responsibilities (W3)	W3	Not Started	docs/w3-architecture	
P1 — End-to-end demo walkthrough + deployment steps (W4)	W4	Not Started	docs/w4-deployment	
P1 — Final report + presentation slides compilation (W4)	W4	Not Started	docs/w4-report-slides	

Figure 1 — P1 full task list across W1–W4

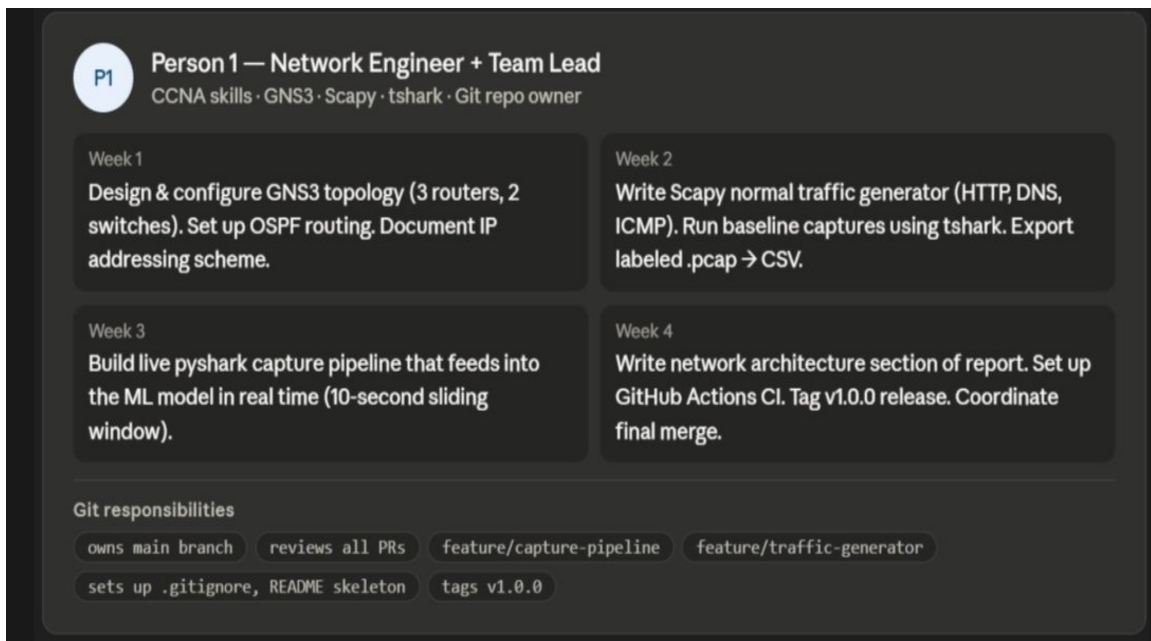


Figure 2 — P1 role card showing weekly responsibilities

3. Task 1 — Project Planning, Role Allocation, Timeline (W1)

Status: ■ Complete

What was done:

- Created a Google Sheets tracker with all 4 teammates' tasks, roles and weekly deliverables
- Documented the two critical blocking dependencies between team members
- Established week-by-week milestones for all 4 people
- Shared tracker link with entire team as single source of truth

Why this was done first:

In any team project, the biggest risk is not technical — it is coordination failure. If two people work on incompatible assumptions, you lose days. A shared tracker aligned everyone on ownership and surfaced the blocking dependencies early so they could be de-risked before becoming emergencies.

Tool chosen: Google Sheets (not Jira/Trello)

Jira is enterprise-grade overhead for a 4-person academic project. Google Sheets is zero setup, everyone already has access, and it is fully auditable. The tool should serve the team, not the other way around.

4. Task 2 — Repository Setup, Branching Strategy, Issue Labels (W1)

Status: ■ Complete — Branch: chore/w1-repo-setup

Steps completed:

Step	What was done
GitHub repo created	Public repo named network-anomaly-detection
Cloned locally	git clone → cd → git checkout -b chore/w1-repo-setup
Folder structure created	capture/ ml/ api/ dashboard/ docs/
.gitkeep files added	Git doesn't track empty folders — .gitkeep preserves them
.gitignore configured	Excludes __pycache__, *.pcap, *.csv, .env, venv/
README.md written	Full project description, pipeline, team roles, tech stack
PR created and merged	chore/w1-repo-setup → main via Pull Request
Branch protection set	No direct pushes to main — all changes via PR

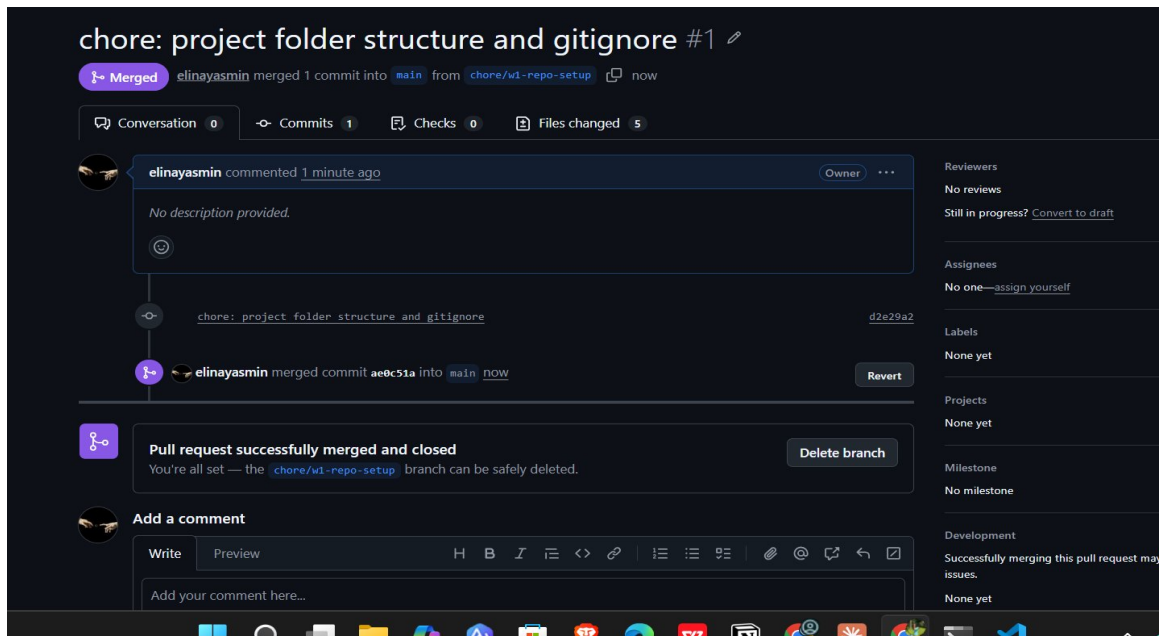


Figure 3 — PR #1 successfully merged into main

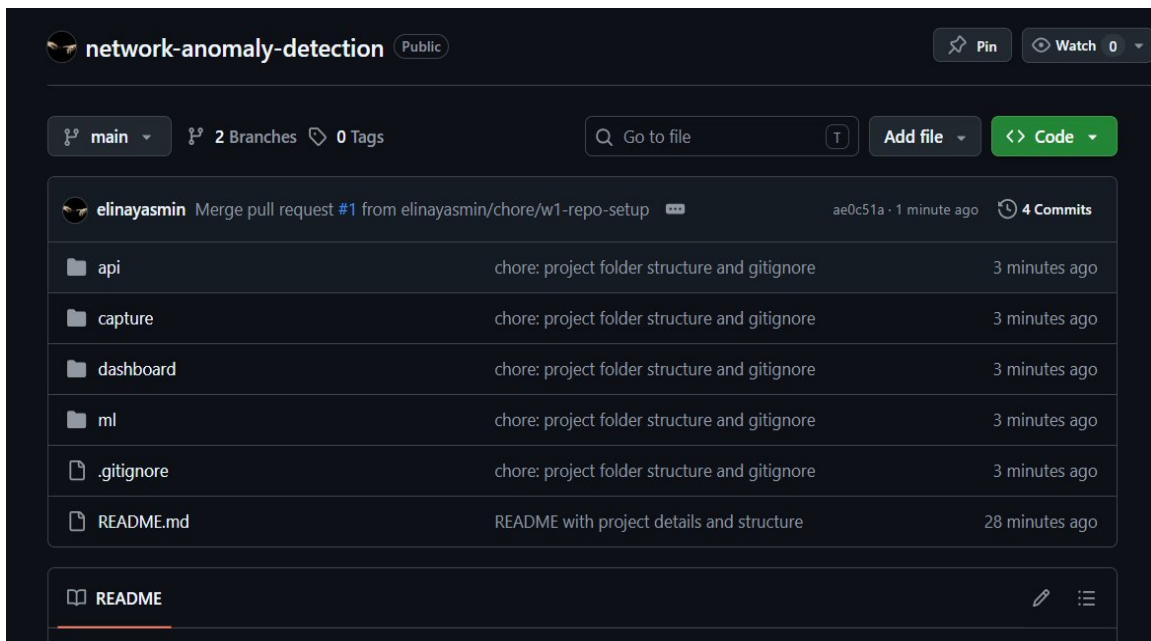


Figure 4 — GitHub repo live on main with full folder structure

Project planning, role allocation, timeline (W1)	W1	Done	NA
Set up repo, branching strategy, issue labels (W1)	W1	Done	chore/w1-repo-setup

Figure 5 — Tracker updated: W1 tasks marked Done

5. Task 3 — Flask API + SQLite Backend (W2)

Status: ■ Complete — Branch: [backend/w2-api-contracts](#)

Built the backbone REST API that connects all system components. The API acts as the middleman — P3's ML model POSTs predictions to it, P4's dashboard GETs anomaly data from it.

API Endpoints:

Endpoint	Method	Who Uses It	Purpose
/api/health	GET	Everyone	Check if API is alive
/api/anomalies	GET	P4 Dashboard	Fetch all logged anomalies
/api/anomalies	POST	P3 ML Model	Log a new anomaly detection

Database Schema (SQLite — *anomalies.db*):

Field	Type	Purpose
id	INTEGER PK	Auto-incremented unique ID
timestamp	TEXT	When anomaly was detected
src_ip	TEXT	Source IP address
dst_ip	TEXT	Destination IP address
protocol	TEXT	HTTP / DNS / ICMP
anomaly_score	REAL	ML confidence score (0.0 – 1.0)
label	TEXT	normal or anomaly

Test Results:

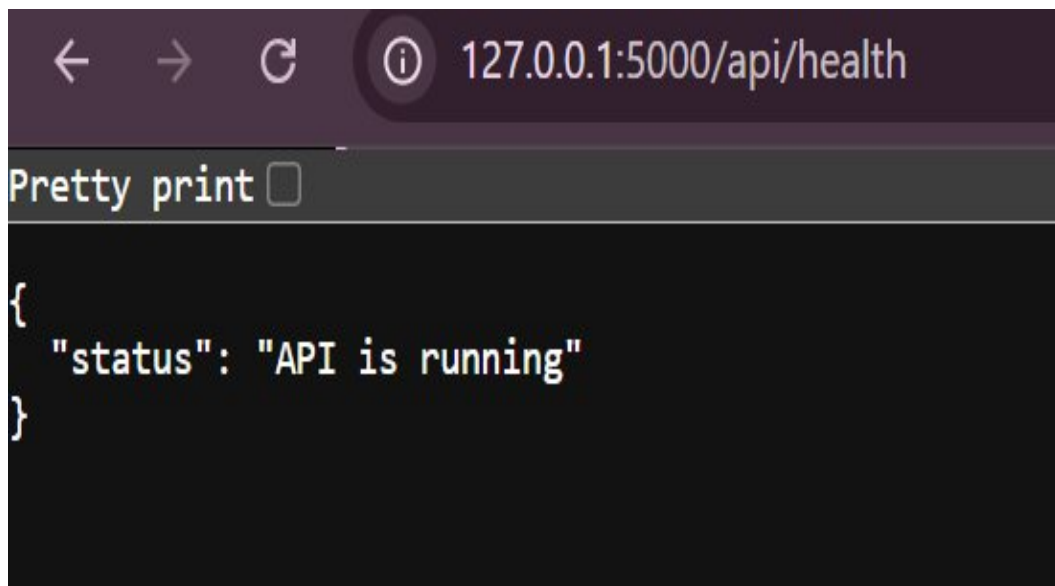


Figure 6 — GET /api/health returning 'API is running'

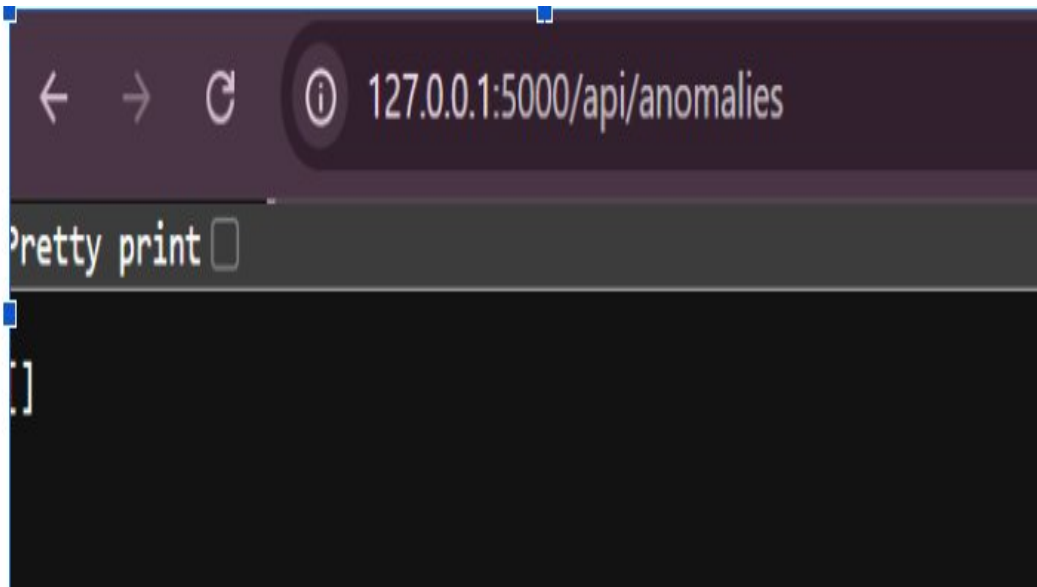


Figure 7 — GET /api/anomalies returning empty list before any data

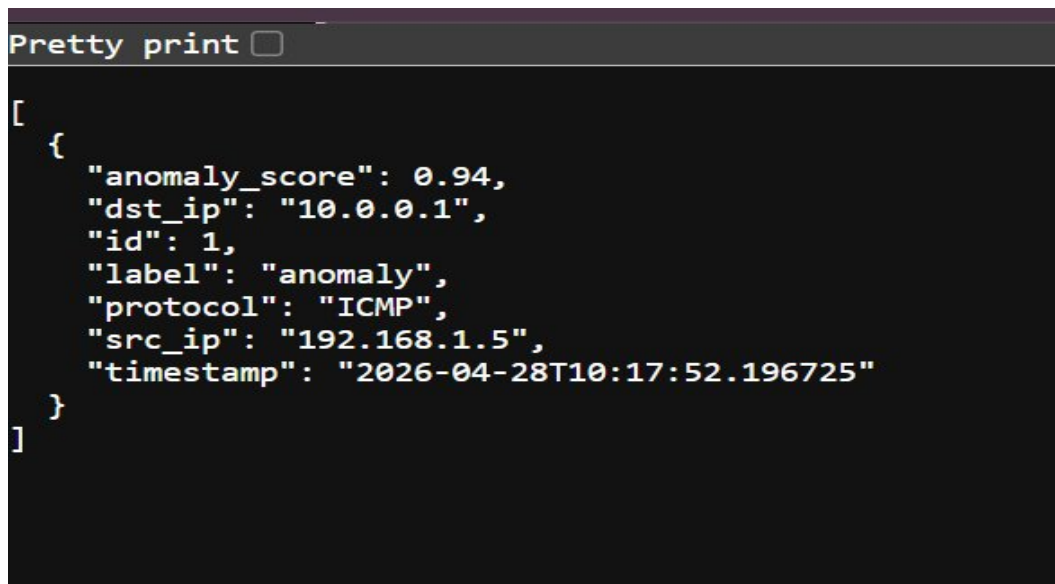


Figure 8 — GET /api/anomalies after POST test — anomaly logged correctly

6. Task 4 — Network Data Capture Pipeline & Topology (W2)

Status: ■ In Progress — Branch: backend/w2-capture-design

Tools Installed:

Tool	Version	Purpose	Status
tshark	4.6.4	Packet capture CLI tool	■ Ready
pyshark	latest	Python packet parsing library	■ Ready
Flask	latest	REST API framework	■ Ready
SQLite3	built-in	Lightweight database	■ Ready
GNS3	2.2.58.1	Installed but not used (PT chosen)	■■ Installed
Cisco Packet Tracer	latest	Network topology simulator	■ Active
Netmiko	pending	Python SSH automation library	■ To install

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\KIIT0001> tshark --version
TShark (Wireshark) 4.6.4 (v4.6.4-0-g93282876538d).

Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Compile-time info:
  Bit width: 64-bit
  Compiler: Microsoft Visual Studio 2022 (VC++ 14.44, build 35223)
  GLib: 2.86.3
With:
  +brotli
  +Gcrypt 1.11.2-unknown
  +GnuTLS 3.8.12 and PKCS#11
  +Kerberos (MIT)
  +libpcap
  +libsmi 0.5.0
  +libxml2 2.15.1
  +Lua 5.4.6 (UFW patched)
  +LZ4 1.10.0
  +MaxMind
  +nghttp2 1.65.0
  +nghttp3 1.8.0
  +PCRE2 10.47 2025-10-21
  +Snappy 1.1.9
  +xxhash 0.8.3
  +zlib 1.3.1
  +zlib-ng 2.2.3
  +Zstandard 1.5.7

Runtime info:
  OS: 64-bit Windows 11 (25H2), build 26200
  CPU: 12th Gen Intel(R) Core(TM) i7-1255U (with SSE4.2)
  Memory: 16092 MB of physical memory
  GLib: 2.86.3
```

Figure 9 — tshark, pyshark and Flask all verified ready

Cisco Packet Tracer Topology — 3 Floor Office Building:

Designed a complex enterprise network topology representing a 3-floor office building with VLSM subnetting, VLANs per floor, OSPF routing across all routers, and wireless access points on each floor.

Device Selection:

Device	Model	Qty	Role	Why This Model
Router	Cisco 2911	4	R1-Edge, R2-Core, R3-Floor1, R4-Floor2	10G SFP ports, enterprise standard
Switch	Catalyst 2960-24TT	4	SW-Floor1/2/3, SW-Servers	Most deployed access layer switch, full VLAN support

Access Point	AccessPoint-PT	3	AP-Floor1/2/3	Wireless coverage per floor
PC	PC-PT	6	IT-PC1/2, HR-PC1/2, FIN-PC1/2	End user workstations
Laptop	Laptop-PT	2	IT-Laptop, FIN-Laptop	Wireless mobile workers
Server	Server-PT	1	DNS-WebServer	DNS + Web services
Printer	Printer-PT	1	HR-Printer	Network printer Floor 2
Cloud	PT-Cloud	1	Cloud0	Internet/ISP simulation

IP Addressing Plan (VLSM):

Network	Subnet	Usable Hosts	Location
WAN Link	203.0.113.0/30	2	Cloud → R1-Edge
Backbone P2P	10.0.0.0/30	2	R1-Edge → R2-Core
Backbone P2P	10.0.0.4/30	2	R2-Core → R3-Floor1
Backbone P2P	10.0.0.8/30	2	R2-Core → R4-Floor2-3
Server Room	192.168.1.0/28	14	VLAN 99 — SW-Servers
Floor 1 IT	192.168.2.0/26	62	VLAN 10 — SW-Floor1
Floor 2 HR	192.168.3.0/27	30	VLAN 20 — SW-Floor2
Floor 3 Finance	192.168.4.0/27	30	VLAN 30 — SW-Floor3

WiFi Configuration:

Access Point	SSID	Channel	Connected Devices
AP-Floor1	Floor1-WiFi	1 (non-overlapping)	IT-Laptop
AP-Floor2	Floor2-WiFi	6 (non-overlapping)	—
AP-Floor3	Floor3-WiFi	11 (non-overlapping)	FIN-Laptop

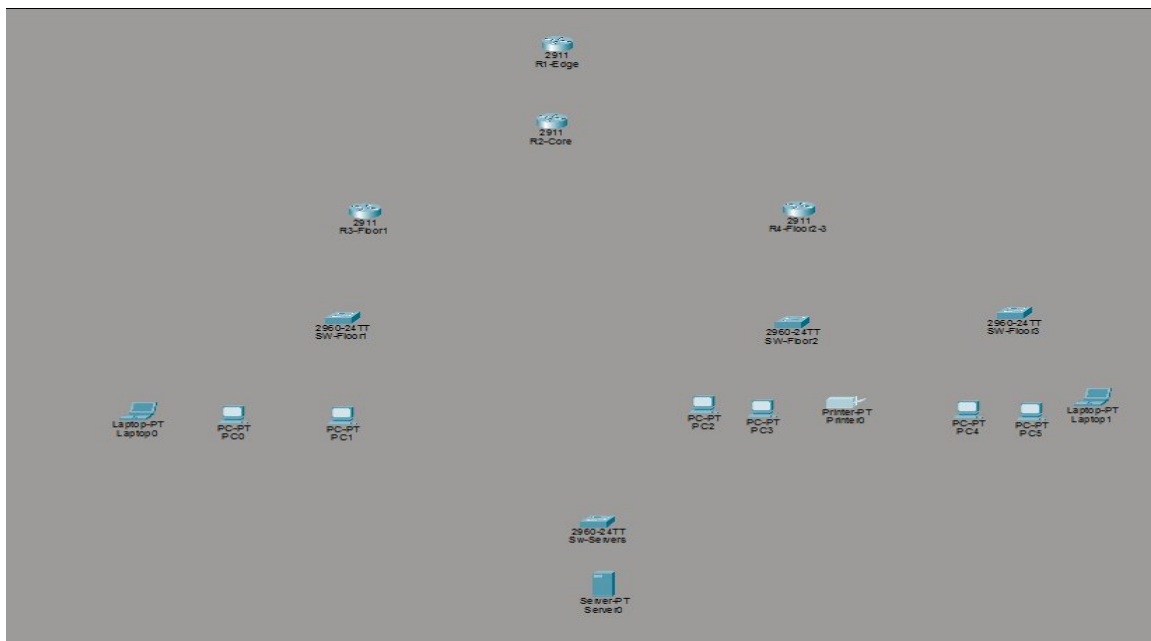


Figure 10 — All devices placed and named on Packet Tracer canvas

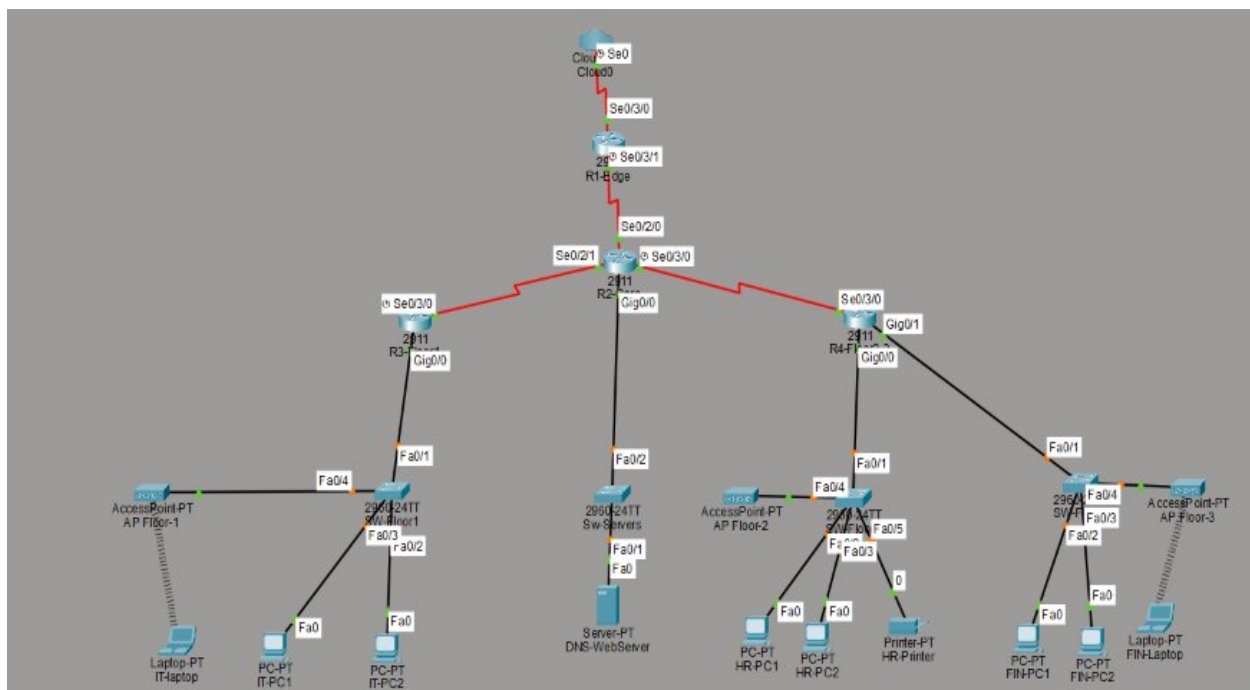


Figure 11 — Full topology with cables connected (IP config pending)

7. Tool Decisions & Rationale

Decision	Chosen	Rejected	Key Reason
Network sim	Cisco Packet Tracer	GNS3	Mastering Cisco tools for Cisco interview
Traffic data	CICIDS dataset	Scapy generation	Real labeled attack traffic for stronger ML
API framework	Flask	Django	Lightweight, no overhead for microservice
Database	SQLite	PostgreSQL	Zero server setup, academic scale sufficient
Automation	Netmiko + DevNet	Manual CLI only	Python SSH automation, real IOS XE device
Router model	Cisco 2911	1941 / 4331	3 GigE ports, ISR G2 enterprise standard
Switch model	Catalyst 2960	3560 / PT-Switch	Access layer standard, full VLAN support
WiFi channels	1, 6, 11	Other channels	Only non-overlapping 2.4GHz channels

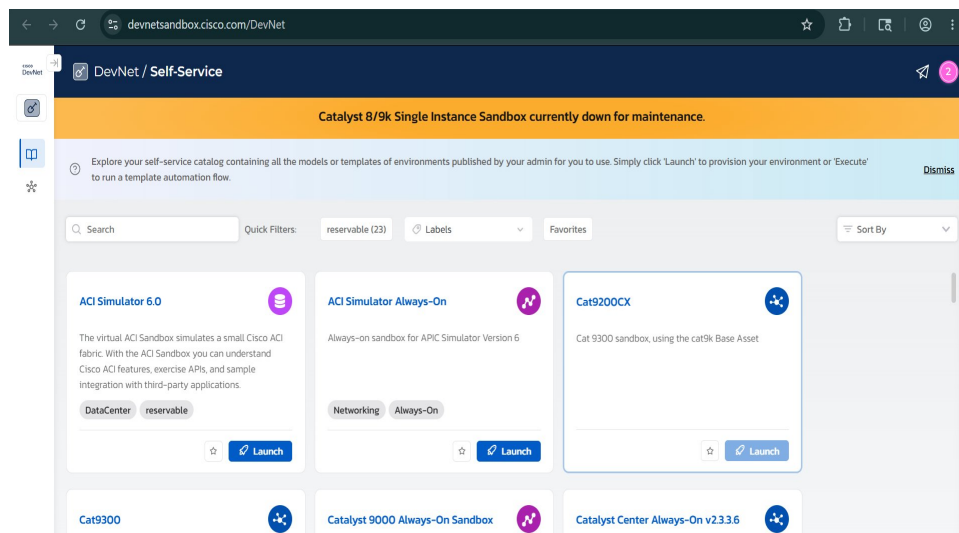


Figure 12 — Cisco DevNet IOS XE on Cat8kv selected for Netmiko automation

8. Failures & Fixes

Failure 1 — mkdir multiple folders failed in PowerShell

What happened	Running 'mkdir capture ml api dashboard docs' gave a ParameterBinding error
Why it happened	PowerShell's mkdir doesn't accept multiple space-separated folder names unlike bash/Linux
How it was fixed	Ran mkdir separately for each folder: mkdir capture, mkdir ml, etc.
Lesson learned	Always adapt Linux commands to PowerShell equivalents on Windows

Failure 2 — touch command not recognized

What happened	'touch .gitignore' failed — 'touch is not recognized as a cmdlet'
Why it happened	touch is a Linux/bash command. PowerShell uses New-Item
How it was fixed	Used: New-Item .gitignore -ItemType File
Lesson learned	PowerShell has different equivalents for almost every bash command

Failure 3 — tshark not found after Wireshark install

What happened	'tshark --version' gave CommandNotFoundException despite Wireshark being installed
Why it happened	Wireshark adds GUI to PATH but not tshark CLI. tshark.exe exists in C:\Program Files\Wireshark but PATH doesn't include it
How it was fixed	Added C:\Program Files\Wireshark to System PATH via Environment Variables
Lesson learned	Always add CLI tools to PATH manually when installed via GUI installers

Failure 4 — GNS3 installer blocked by Windows SmartScreen

What happened	'Windows protected your PC' warning appeared when running GNS3 installer
Why it happened	SmartScreen flags any installer not from Microsoft Store
How it was fixed	Clicked 'More info' → 'Run anyway' — GNS3 is official open source software
Lesson learned	SmartScreen is cautious not a definitive block — always verify software source

Failure 5 — Npcap downgrade prompt during GNS3 install

What happened	GNS3 installer asked to replace Npcap 1.83 with 1.78
Why it happened	GNS3 bundled an older Npcap version, machine already had newer one
How it was fixed	Clicked No — never replace newer version with older
Lesson learned	Always keep the newer version of dependencies

Failure 6 — Branch name with space caused fatal Git error

What happened	'git checkout -b backend/w2-capture design' — fatal: 'design' is not a commit
Why it happened	Branch names cannot contain spaces. Git read 'design' as a separate argument
How it was fixed	Reran with hyphen: git checkout -b backend/w2-capture-design
Lesson learned	Branch names always use hyphens, never spaces

Failure 7 — Access Points cross-connecting wirelessly

What happened	AP-Floor1 and AP-Floor2 showed dotted connection line to each other
Why it happened	Both APs had same default SSID and channel — they associated with each other
How it was fixed	Changed each AP to unique SSID (Floor1-WiFi, Floor2-WiFi, Floor3-WiFi) and non-overlapping channels (1, 6,
Lesson learned	Always use non-overlapping WiFi channels in multi-AP environments

Failure 8 — Packet Tracer cannot be automated with Netmiko

What happened	Planned to use Netmiko to automate router config in Packet Tracer
Why it happened	Packet Tracer is a closed simulator — it doesn't expose real SSH ports to host machine
How it was fixed	Used Cisco DevNet Sandbox (IOS XE on Cat8kv AlwaysOn) — real Cisco device accessible via SSH
Lesson learned	Packet Tracer = visual learning only. Real automation needs real SSH access

```
Windows PowerShell
PS C:\Users\KIIT0001\network-anomaly-detection> git checkout main
M
.gitignore
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\KIIT0001\network-anomaly-detection> git checkout main
M
.gitignore
Already on 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\KIIT0001\network-anomaly-detection> git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 930 bytes | 66.00 KiB/s, done.
From https://github.com/elinayasmin/network-anomaly-detection
* branch          main          -> FETCH_HEAD
   ae0c51a..a97f8e6  main        -> origin/main
Updating ae0c51a..a97f8e6
Fast-forward
 anomalies.db | Bin 0 -> 12288 bytes
 api/app.py   | 75 +++++
 2 files changed, 75 insertions(+)
 create mode 100644 anomalies.db
 create mode 100644 api/app.py
PS C:\Users\KIIT0001\network-anomaly-detection>
PS C:\Users\KIIT0001\network-anomaly-detection> git checkout -b backened/w2-capture design
fatal: 'design' is not a commit and a branch 'backened/w2-capture' cannot be created from it
PS C:\Users\KIIT0001\network-anomaly-detection> |
```

Figure 13 — Branch name space error and correct fix applied

9. Interview Answer Framework (Cisco Interview Prep)

Answer Template:

Every technical answer follows this structure: (1) What was done → (2) Technical reason → (3) Alternative considered → (4) Why alternative didn't fit → (5) Real world justification.

Q: Why Cisco Packet Tracer over GNS3?

Packet Tracer is Cisco's official learning platform and I wanted to deepen Cisco-specific skills for professional development. GNS3 is excellent for advanced emulation but uses community-sourced IOS images. For a Cisco-focused career path, mastering the official toolset was the priority. In a professional environment with CML licensing, I would choose CML for its tighter IOS integration.

Q: Why Cisco 2911 routers?

The 2911 is the industry standard ISR G2 router for enterprise branch offices. It has 3 built-in GigabitEthernet interfaces which matched our topology — the core router needed to connect simultaneously to edge, floor 1, and floors 2 and 3. The 1941 only has 2 GigE ports which would have forced compromises in the design.

Q: Why Catalyst 2960 switches?

The 2960 is the most widely deployed access layer switch in real enterprise environments. It fully supports 802.1Q VLANs and trunking which our multi-floor VLAN design required. We deliberately avoided Layer 3 switches like the 3560 because inter-VLAN routing was handled at the router level — keeping the design true to a real routed interface architecture.

Q: Why CICIDS dataset instead of generating traffic?

The CICIDS dataset from the Canadian Institute for Cybersecurity contains real labeled network captures including DoS, port scan, and brute force scenarios. This gave our ML model exposure to realistic attack patterns rather than simplified synthetic traffic. It is also used in published research papers which gives our project academic credibility.

Q: Why Flask over Django?

Flask is lightweight and perfect for an internal API that only needs to receive ML predictions and serve them to a dashboard. Django's ORM, admin panel, and full-stack features added overhead that was not justified for our microservice use case. The real value was defining clear API contracts early so P3 and P4 could work independently without waiting on each other.

Q: Why non-overlapping WiFi channels 1, 6, 11?

In the 2.4GHz band, channels 1, 6 and 11 are the only truly non-overlapping channels. Using overlapping channels between adjacent access points causes interference which degrades throughput. Assigning one non-overlapping channel per floor is standard enterprise wireless design practice.

Killer Closing Statement:

"This project sits at the intersection of networking, data science and AI. The future extension I am most excited about is using Netmiko to automatically push ACL rules to the router the moment the ML model detects an anomaly — closing the loop from AI detection to automated network response. That is exactly how modern SOC automation works in enterprise environments."

10. Remaining Tasks

Task	Week	Branch	Status
Finish PT topology — IP config + OSPF + VLANs	W2	backend/w2-capture-design	■ Next
Write Netmiko automation script (DevNet)	W2	backend/w2-capture-design	■ Pending
Download + clean CICIDS dataset (P2)	W2	—	■ P2 task
Backend integration — pyshark + CICIDS → API	W3	backend/w3-integration	■ Pending
System architecture diagram (draw.io)	W3	docs/w3-architecture	■ Pending
End-to-end demo + deployment steps	W4	docs/w4-deployment	■ Pending
Final report + presentation slides	W4	docs/w4-report-slides	■ Pending

Immediate Next Steps:

1. Configure IP addresses on all 4 routers via Packet Tracer CLI
2. Configure OSPF Area 0 on all routers
3. Configure VLANs 10, 20, 30, 99 on all switches
4. Reserve IOS XE on Cat8kv on Cisco DevNet Sandbox
5. pip install netmiko
6. Write Netmiko Python script to automate router configuration
7. Commit and push topology file to backend/w2-capture-design branch
8. Open PR and merge to main